

Object Inheritance Lab Part One (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p3-inheritance-lab>  <https://github.com/learn-co-curriculum/python-p3-inheritance-lab/issues/new>

Learning Goals

- Reduce repeated code and enhance objects using inheritance.
 - Define classes that inherit from a shared parent, or superclass.
 - Define methods unique to those classes.
 - Define methods in the child class that use the `super` keyword to inherit from and augment methods in the parent class.
- Accomplish complex programming tasks using knowledge from previous modules.

Key Vocab

- **Inheritance**: a tool that allows us to recycle code by creating a class that "inherits" the attributes and methods of a parent class.
- **Composition**: a tool that enables you to recycle code by adding objects to other objects. Rather than building on a base class as in inheritance, composition leverages the attributes and methods of an instance of another class.
- **Subclass**: a class that inherits from another class. Colloquially called a "child" class.
- **Superclass**: a class that is inherited by another class. Colloquially called a "parent" class.
- **Child**: another name for a subclass.
- **Parent**: another name for a superclass.
- `super()`: a built-in Python function that allows us to manipulate the attributes and methods of a superclass from the body of its subclass.
- **Decorator**: syntax that allows us to add functionality to an object without modifying its structure.

Introduction

In this lab, we'll be working with a school domain model. Our application has users that are either teachers or students. Teachers and students will share certain attributes and have certain behaviors that are unique to them. You'll be defining a `User` class that both students and teachers inherit from and you'll be writing methods within both the `Teacher` and `Student` class that are unique to that class.

Instructions

This is a test-driven lab. Because "Student" comes before "Teacher" alphabetically and we're aiming to test them in the opposite order, start off with `pytest lib/testing/user_test.py` . (If you're already in `lib/` , leave it out). Continue this process for each tested class. And submit your work when complete.

1. Define the `User` class such that a user is instantiated with a `first_name` and `last_name` . These should be saved as attributes.
2. We've given you a barebones `Teacher` class in `lib/teacher.py` . Change the class definition so that the `Teacher` class inherits from the `User` class. Run the test suite and notice that you are passing some tests for the `Teacher` class, even without writing any code inside that class. That is because it will inherit the `first_name` and `last_name` attributes from the `User` class you told it to inherit from. We've given you a list of knowledge strings below; modify the `Teacher` class so that it initializes with this list stored as an attribute, `self.knowledge` .

```
knowledge = [  
    "str is a data type in Python",  
    "programming is hard, but it's worth it",  
    "JavaScript async web request",  
    "Python function call definition",  
    "object-oriented teacher instance",  
    "programming computers hacking learning terminal",  
    "pipenv install pipenv shell",  
    "pytest -x flag to fail fast",  
]
```

3. Expand the `teach()` method in the `Teacher` class so that it returns a random element from `self.knowledge`. We have imported [Python's random library](https://docs.python.org/3/library/random.html) [↗\(https://docs.python.org/3/library/random.html\)](https://docs.python.org/3/library/random.html) to assist you. You will want to use the `random.randint()` method to choose a random index in `self.knowledge`. This method takes two arguments, a minimum number and a maximum number, and returns a random element in the range.
4. We've given you a barebones `Student` class. Change the class definition so that it inherits from the `User` class. Run the test suite and notice that you are passing some tests for the `Student` class, even without writing any code inside that class. That is because it will inherit the `first_name` and `last_name` attributes from the `User` class you told it to inherit from.
5. Individual students should initialize with an attribute, `self.knowledge`, that points to an empty list.
6. Expand the `learn()` method in the `Student` class so that it takes in a string and adds that string to the student's `self.knowledge` list.

Conclusion

We've seen how to set up inheritance to share behavior from one class to another using parent classes as arguments in our class definition (`class Child(Parent)`), which lets the subclass use attributes and methods that are defined on the parent class. We also discussed how **class introspection** works in Python, when multiple classes define the same method.

Resources

- [Python 3.8 Documentation](https://docs.python.org/3.8/) [↗\(https://docs.python.org/3.8/\)](https://docs.python.org/3.8/)
- [Inheritance - Python](https://docs.python.org/3/tutorial/classes.html#inheritance) [↗\(https://docs.python.org/3/tutorial/classes.html#inheritance\)](https://docs.python.org/3/tutorial/classes.html#inheritance)
- [Inheritance and Composition: A Python OOP Guide - Real Python](https://realpython.com/inheritance-composition-python/) [↗\(https://realpython.com/inheritance-composition-python/\)](https://realpython.com/inheritance-composition-python/)
- [Python Metaclasses - Real Python](https://realpython.com/python-metaclasses) [↗\(https://realpython.com/python-metaclasses\)](https://realpython.com/python-metaclasses)

This tool needs to be loaded in a new browser window

Load Object Inheritance Lab Part One (CodeGrade) in a new window

